A report on

Forecast passenger traffic for airports using historical flight and travel data.

Project Submitted By		
Name	**Reg. No.**	**Roll Number**
Shaik Sabeer Hussain	**12322992**	**27**

Submitting to

Amanpal Singh Rayat

S.no	topics	Pg.no
1	Abstract	2
2	introduction	2-3
3	Objectives	4
4	methodology	4-5
5	working	6
6	Software used	6
7	code	7-8
8	output	9-10
9	Advantages and disadvantages	11
10	applications	11
11	Conclusion	11
12	Future scope	12

1. Abstract

This project focuses on predicting airport passenger traffic using historical air traffic data and machine learning techniques implemented in PySpark. The dataset contains information such as operating airline, passenger count, year, and month. The primary objective is to analyze trends in passenger flow and develop a predictive model to estimate future passenger volumes.

Initially, the dataset is loaded into a PySpark DataFrame, where data preprocessing techniques such as handling missing values, type conversion, and filtering inconsistent records are applied. Feature engineering is performed by transforming categorical variables like airline names into numerical indices using encoding techniques, and extracting meaningful features such as month and year.

A Linear Regression model from PySpark MLlib is then trained on the processed data to predict passenger count. The model performance is evaluated using metrics such as Root Mean Square Error (RMSE) and R-squared (R^2), providing insights into prediction accuracy. Additionally, visualization techniques are used to compare actual and predicted passenger traffic trends.

The results indicate that while Linear Regression provides a baseline model, the accuracy can be improved by incorporating additional features and advanced algorithms. This project demonstrates the effectiveness of PySpark in handling large-scale data processing and building scalable machine learning models for real-world applications like airport traffic forecasting.

2. Introduction

2.1 Background

With the rapid growth of data generation in the aviation industry, analyzing and predicting airport passenger traffic has become increasingly important. Airports handle millions of passengers every year, and efficient management of this traffic is essential for improving operational efficiency, reducing congestion, and enhancing passenger experience. Traditional methods of analyzing airport traffic rely heavily on manual observation and basic statistical techniques, which are often insufficient for handling large-scale and dynamic datasets.

To address these challenges, big data technologies such as Apache Spark provide a scalable and efficient solution for processing and analyzing large volumes of data. PySpark, the Python API for Spark, enables distributed data processing and machine learning, making it suitable for handling complex datasets like airport traffic records. In this project, historical air traffic data is used to predict passenger volume using a Linear Regression model implemented in PySpark.

The system processes data such as airline operations, passenger counts, and time-related attributes (year and month) to identify patterns and trends. By leveraging machine learning techniques, the project aims to forecast passenger traffic, which can assist airport authorities in better planning and resource allocation.

2.2 Importance of Airport Traffic Prediction

In modern aviation systems, accurate prediction of passenger traffic is crucial for efficient airport management. High passenger volumes can lead to congestion, delays, and poor service quality, while underutilization of resources can result in operational inefficiencies. Therefore, predicting passenger traffic helps in optimizing resource allocation, improving scheduling, and enhancing overall airport performance.

Machine learning-based prediction systems provide a more reliable and data-driven approach compared to traditional methods. By analyzing historical trends, these systems can forecast future passenger volumes with reasonable accuracy. This helps airport authorities in making informed decisions related to staffing, infrastructure usage, and security management.

Additionally, such predictive systems contribute to the development of smart transportation systems and data-driven decision-making processes. The use of PySpark allows handling large datasets efficiently, making the system scalable and suitable for real-world applications.

2.3 Related Concepts and Technologies

The foundation of this project lies in the integration of big data processing and machine learning techniques. PySpark, a component of Apache Spark, is used for distributed data processing and analysis. It provides powerful tools such as Spark DataFrames for handling structured data and MLlib for implementing machine learning algorithms.

In this project, data preprocessing techniques such as handling missing values, type conversion, and feature engineering are applied using PySpark functions. Categorical data, such as airline names, are converted into numerical form using encoding techniques like StringIndexer. Feature vectors are then created using VectorAssembler to prepare the data for model training.

A Linear Regression model is used to predict passenger count based on selected features such as year, month, and airline index. The model is evaluated using performance metrics like Root Mean Square Error (RMSE) and R-squared (R^2), which help in assessing prediction accuracy.

This project demonstrates how big data tools and machine learning can be combined to solve real-world problems in the aviation domain. It also highlights the role of data analytics in building intelligent systems for forecasting and decision-making, contributing to the advancement of smart transportation and data-driven infrastructure management.

3. Objectives

- To develop a system for predicting airport passenger traffic using historical data.
- To utilize PySpark for efficient handling and processing of large-scale datasets.
- To perform data preprocessing, including handling missing values, type conversion, and data cleaning.
- To apply feature engineering techniques such as extracting and transforming relevant attributes (e.g., year, month, airline).
- To convert categorical data into numerical form using encoding methods like StringIndexer.
- To build a Linear Regression model using PySpark MLlib for predicting passenger count.
- To evaluate the model performance using metrics such as RMSE (Root Mean Square Error) and R^2 score.
- To visualize and analyze trends in passenger traffic for better understanding and interpretation.
- To demonstrate the use of big data and machine learning techniques in real-world aviation applications.
- To provide a scalable and efficient solution that can support data-driven decision-making in airport management.

4. Methodology

4.1 Methodology

The methodology of this project involves a sequence of steps including data collection, preprocessing, feature engineering, model building, and evaluation using PySpark. The entire process is designed to handle large datasets efficiently and generate meaningful predictions of airport passenger traffic.

1. Data Collection

   - The dataset used in this project consists of historical airport traffic data containing attributes such as:
   - Operating Airline
   - Passenger Count
   - Year
   - Month
   - This data is stored in a CSV file and loaded into a PySpark DataFrame for further processing.

2. Data Preprocessing

   Data preprocessing is performed to ensure the dataset is clean and suitable for analysis. The following steps are applied:

   - Removal of missing or null values in the passenger count column
   - Conversion of data types (e.g., passenger count to integer, year to numeric format)
   - Filtering of invalid or inconsistent data

   These steps help improve the quality of the dataset and ensure accurate model performance.

3. Feature Engineering

Feature engineering is used to transform raw data into meaningful inputs for the model:

- The Month column is converted from text format (e.g., January) to numerical values (1–12)
- The Operating Airline (categorical data) is encoded into numerical form using StringIndexer
- Relevant features such as Year, Month, and Airline Index are selected

This step ensures that all features are in a format suitable for machine learning algorithms.

4. Feature Vector Creation

Using VectorAssembler, selected features are combined into a single feature vector column. This is required because PySpark ML models accept input in vector format.

5. Model Building

A Linear Regression model is built using PySpark MLlib:

- The dataset is split into training (80%) and testing (20%) sets
- The model is trained on the training dataset
- The trained model learns relationships between input features and passenger count

6. Model Evaluation

The model performance is evaluated using:

- RMSE (Root Mean Square Error) → measures prediction error
- R^2 Score → measures how well the model fits the data

These metrics help in understanding the accuracy and reliability of the model.

7. Visualization

The results are visualized by plotting:

- Actual passenger count
- Predicted passenger count

This helps in comparing model predictions with real data and understanding trends.

5.Working of the System

The working of the system can be explained step-by-step as follows:

Step 1: Initialize Spark Environment
The system begins by creating a **SparkSession**, which acts as the entry point for all PySpark operations.

Step 2: Load Dataset
The CSV file containing airport traffic data is loaded into a **PySpark DataFrame** for processing.

Step 3: Clean the Data
The system removes null values and converts columns into appropriate data types to ensure consistency.

Step 4: Transform Features

- Month names are converted into numerical values
- Airline names are encoded into numeric indices
- Required features are selected

Step 5: Prepare Input for Model
The selected features are combined into a single vector using **VectorAssembler**.

Step 6: Train the Model
The Linear Regression model is trained using the training dataset.

Step 7: Generate Predictions
The trained model predicts passenger count for the test dataset.

Step 8: Evaluate Results
The system calculates RMSE and R^2 score to measure model performance.

Step 9: Display Output
The system displays:

- Actual vs Predicted values
- Performance metrics
- Graphical visualization

6. Software Used

IDLE to upload code and powershell to see the output.
I have used pyspark coding for this project

7. code

Airport_traffic.py - C:\Users\my\OneDrive\Desktop\Airport Traffic project\Airport_traffic.py (3.10.0)

File Edit Format Run Options Window Help

```python
# ------------------------------
# IMPORT LIBRARIES
# ------------------------------
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, when
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator
import matplotlib.pyplot as plt

# ------------------------------
# START SPARK SESSION
# ------------------------------
spark = SparkSession.builder.appName("Airport Traffic Prediction").getOrCreate()

print("\n===== AIRPORT TRAFFIC PREDICTION =====\n")

# ------------------------------
# LOAD DATASET
# ------------------------------
df = spark.read.csv("air_traffic_data.csv", header=True, inferSchema=True)

print("Dataset Columns:")
print(df.columns)

print("\nSample Data:")
df.show(5, truncate=False)

# ------------------------------
# DATA CLEANING
# ------------------------------
df = df.dropna(subset=["Passenger Count"])
df = df.withColumn("Passenger Count", col("Passenger Count").cast("int"))
df = df.withColumn("Year", col("Year").cast("int"))

print("\nData after cleaning:")
df.select("Passenger Count", "Year", "Month").show(5)

# ------------------------------
# FEATURE ENGINEERING
# ------------------------------
df = df.withColumn("Month_num",
    when(col("Month")=="January",1)
    .when(col("Month")=="February",2)
    .when(col("Month")=="March",3)
```

Ln: 19 Col: 14

Airport_traffic.py - C:\Users\my\OneDrive\Desktop\Airport Traffic project\Airport_traffic.py (3.10.0)

File Edit Format Run Options Window Help

```python
# ------------------------------
df = df.withColumn("Month_num",
    when(col("Month")=="January",1)
    .when(col("Month")=="February",2)
    .when(col("Month")=="March",3)
    .when(col("Month")=="April",4)
    .when(col("Month")=="May",5)
    .when(col("Month")=="June",6)
    .when(col("Month")=="July",7)
    .when(col("Month")=="August",8)
    .when(col("Month")=="September",9)
    .when(col("Month")=="October",10)
    .when(col("Month")=="November",11)
    .when(col("Month")=="December",12)
)

# Encode Airline
indexer = StringIndexer(inputCol="Operating Airline", outputCol="airline_index")
df = indexer.fit(df).transform(df)

print("\nFeature Engineered Data:")
df.select("Year", "Month_num", "airline_index").show(5)

# ------------------------------
# FEATURE VECTOR
# ------------------------------
assembler = VectorAssembler(
    inputCols=["Year", "Month_num", "airline_index"],
    outputCol="features"
)

data = assembler.transform(df)

# ------------------------------
# TRAIN MODEL
# ------------------------------
train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)

lr = LinearRegression(
    featuresCol="features",
    labelCol="Passenger Count"
)

model = lr.fit(train_data)
```

Ln: 19 Col: 14

Airport_traffic.py - C:\Users\my\OneDrive\Desktop\AIrport Traffic project\Airport_traffic.py (3.10.0)

File Edit Format Run Options Window Help

```python
print("\nModel Training Completed")

# ------------------------------
# PREDICTIONS
# ------------------------------
predictions = model.transform(test_data)

print("\n===== PREDICTION RESULTS =====")

# Clean table
predictions.select(
    col("Passenger Count").alias("Actual"),
    col("prediction").alias("Predicted")
).show(20, truncate=False)

# ------------------------------
# EVALUATION
# ------------------------------
evaluator_rmse = RegressionEvaluator(
    labelCol="Passenger Count",
    predictionCol="prediction",
    metricName="rmse"
)

evaluator_r2 = RegressionEvaluator(
    labelCol="Passenger Count",
    predictionCol="prediction",
    metricName="r2"
)

rmse = evaluator_rmse.evaluate(predictions)
r2 = evaluator_r2.evaluate(predictions)

print("\n===== MODEL PERFORMANCE =====")
print("RMSE (Error):", round(rmse, 2))
print("R2 Score (Accuracy):", round(r2, 4))

# ------------------------------
# VISUALIZATION
# ------------------------------
pdf_pred = predictions.select("Passenger Count", "prediction").toPandas()

print("\nSample Prediction Table:")
print(pdf_pred.head(10))
```

Ln: 19 Col: 14

Airport_traffic.py - C:\Users\my\OneDrive\Desktop\AIrport Traffic project\Airport_traffic.py (3.10.0)

File Edit Format Run Options Window Help

```python
# EVALUATION
# ------------------------------
evaluator_rmse = RegressionEvaluator(
    labelCol="Passenger Count",
    predictionCol="prediction",
    metricName="rmse"
)

evaluator_r2 = RegressionEvaluator(
    labelCol="Passenger Count",
    predictionCol="prediction",
    metricName="r2"
)

rmse = evaluator_rmse.evaluate(predictions)
r2 = evaluator_r2.evaluate(predictions)

print("\n===== MODEL PERFORMANCE =====")
print("RMSE (Error):", round(rmse, 2))
print("R2 Score (Accuracy):", round(r2, 4))

# ------------------------------
# VISUALIZATION
# ------------------------------
pdf_pred = predictions.select("Passenger Count", "prediction").toPandas()

print("\nSample Prediction Table:")
print(pdf_pred.head(10))

# Add error column
pdf_pred["Error"] = pdf_pred["Passenger Count"] - pdf_pred["prediction"]

print("\nWith Error:")
print(pdf_pred.head(10))

# Plot Actual vs Predicted
plt.figure()
plt.plot(pdf_pred["Passenger Count"].values[:50], label="Actual")
plt.plot(pdf_pred["prediction"].values[:50], label="Predicted")
plt.legend()
plt.title("Actual vs Predicted Passenger Traffic")
plt.xlabel("Samples")
plt.ylabel("Passengers")
plt.show()
```

Ln: 19 Col: 14

8.output

```
C:\WINDOWS\system32\cmd.

Dataset Columns:
['Activity Period', 'Operating Airline', 'Operating Airline IATA Code', 'Published Airline', 'Published Airline IATA Code', 'GEO Summ
ary', 'GEO Region', 'Activity Type Code', 'Price Category Code', 'Terminal', 'Boarding Area', 'Passenger Count', 'Adjusted Activity T
ype Code', 'Adjusted Passenger Count', 'Year', 'Month']

Sample Data:
+---------------+-----------------+-------------------------+-----------------+-------------------------+-------------+----------
+-----------------+-----------------+----------+-------------+---------------+-------------------------+-----------------------
+----+-----+
|Activity Period|Operating Airline|Operating Airline IATA Code|Published Airline|Published Airline IATA Code|GEO Summary  |GEO Region
|Activity Type Code|Price Category Code|Terminal  |Boarding Area|Passenger Count|Adjusted Activity Type Code|Adjusted Passenger Count
|Year|Month|
+---------------+-----------------+-------------------------+-----------------+-------------------------+-------------+----------
+-----------------+-----------------+----------+-------------+---------------+-------------------------+-----------------------
+----+-----+
|200507         |ATA Airlines     |TZ                       |ATA Airlines     |TZ                       |Domestic     |US
|Deplaned          |Low Fare          |Terminal 1|B            |27271          |Deplaned                 |27271
|2005|July |
|200507         |ATA Airlines     |TZ                       |ATA Airlines     |TZ                       |Domestic     |US
|Enplaned          |Low Fare          |Terminal 1|B            |29131          |Enplaned                 |29131
|2005|July |
|200507         |ATA Airlines     |TZ                       |ATA Airlines     |TZ                       |Domestic     |US
|Thru / Transit    |Low Fare          |Terminal 1|B            |5415           |Thru / Transit * 2       |10830
|2005|July |
|200507         |Air Canada       |AC                       |Air Canada       |AC                       |International|Canada
|Deplaned          |Other             |Terminal 1|B            |35156          |Deplaned                 |35156
|2005|July |
|200507         |Air Canada       |AC                       |Air Canada       |AC                       |International|Canada
|Enplaned          |Other             |Terminal 1|B            |34090          |Enplaned                 |34090
|2005|July |
+---------------+-----------------+-------------------------+-----------------+-------------------------+-------------+----------
+-----------------+-----------------+----------+-------------+---------------+-------------------------+-----------------------
+----+-----+
only showing top 5 rows
```

```
C:\WINDOWS\system32\cmd.

Data after cleaning:
+---------------+----+-----+
|Passenger Count|Year|Month|
+---------------+----+-----+
|          27271|2005| July|
|          29131|2005| July|
|           5415|2005| July|
|          35156|2005| July|
|          34090|2005| July|
+---------------+----+-----+
only showing top 5 rows


Feature Engineered Data:
+----+---------+-------------+
|Year|Month_num|airline_index|
+----+---------+-------------+
|2005|        7|         48.0|
|2005|        7|         48.0|
|2005|        7|         48.0|
|2005|        7|          5.0|
|2005|        7|          5.0|
+----+---------+-------------+
only showing top 5 rows

26/04/07 19:03:19 WARN Instrumentation: [9772ed55] regParam is zero, which might cause numerical instability and overfitting.
26/04/07 19:03:21 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.blas.JNIBLAS
26/04/07 19:03:21 WARN InstanceBuilder: Failed to load implementation from:dev.ludovic.netlib.lapack.JNILAPACK

Model Training Completed

===== PREDICTION RESULTS =====
+------+-------------------+
```

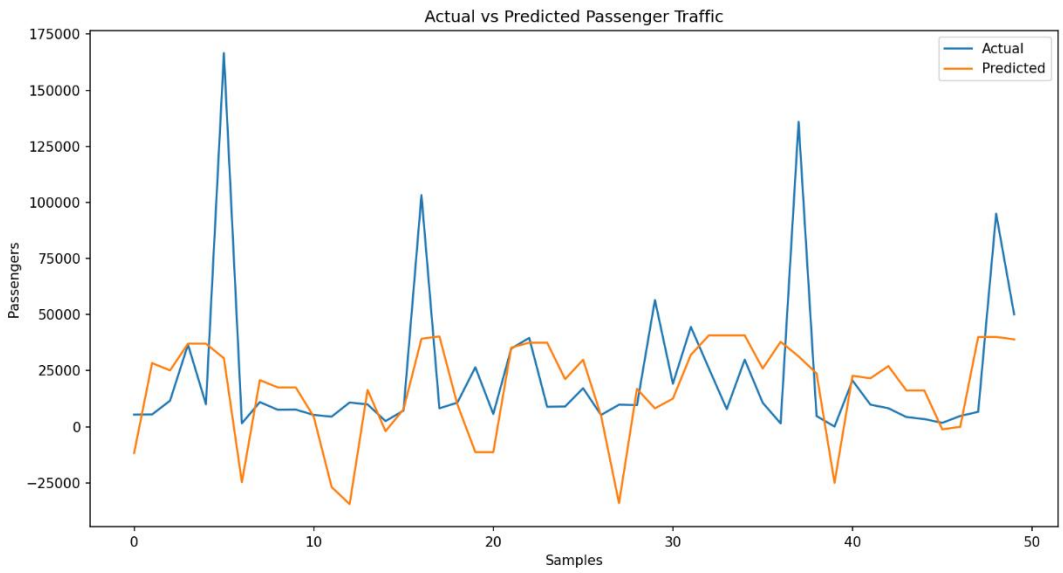
```
C:\WINDOWS\system32\cmd.

Model Training Completed

===== PREDICTION RESULTS =====
+------+-------------------+
|Actual|Predicted          |
+------+-------------------+
|5415  |-11702.295423047617|
|5500  |28383.314556352794 |
|11638 |25133.129963428248 |
|36641 |37050.47347081732  |
|10046 |37050.47347081732  |
|166577|30550.104284968693 |
|1552  |-24703.03379474487 |
|11010 |20799.550506195985 |
|7579  |17549.36591327144  |
|7658  |17549.36591327144  |
|5284  |4548.627541574184  |
|4540  |-26869.823523361236|
|10843 |-34453.587573518045|
|9978  |16465.971048963256 |
|2535  |-1951.7416442744434|
|7229  |7798.812134498265  |
|103279|39217.263199433684 |
|8234  |40300.65806374187  |
|10771 |9965.601863114629  |
|26535 |-11285.563159168232|
+------+-------------------+
only showing top 20 rows


===== MODEL PERFORMANCE =====
RMSE (Error): 57507.03
R2 Score (Accuracy): 0.0809
```

Figure 1



9. Advantages and Disadvantages

Advantages:

- Handles large-scale airport data efficiently using PySpark.
- Faster data processing due to in-memory computation.
- Provides automated prediction of passenger traffic.
- Helps in better planning and decision-making for airport management.
- Scalable system — can handle increasing data in future.
- Easy to implement using Python and PySpark libraries.
- Visualization helps in understanding traffic trends clearly.

Disadvantages:

- Linear Regression model gives low accuracy for complex data.
- Limited features used (only year, month, airline).
- Can produce unrealistic predictions (e.g., negative passenger values).
- Requires proper setup of PySpark and Java environment.
- Not suitable for real-time data processing.
- Performance depends heavily on data quality.

10. Applications

- Helps airport authorities in predicting passenger traffic for better planning.
- Used for resource allocation such as staff management and terminal usage.
- Assists in reducing congestion and delays at airports.
- Supports flight scheduling and operational planning.
- Useful for airline companies to analyze passenger trends.
- Can be applied in smart transportation systems.
- Helps in data-driven decision making in aviation industry.
- Can be extended for real-time traffic prediction systems.

Conclusion

This project successfully demonstrates the application of PySpark and machine learning techniques for predicting airport passenger traffic using historical data. The dataset was processed using PySpark for efficient handling of large-scale data, followed by data cleaning, feature engineering, and model building using Linear Regression. The system is capable of generating predictions and providing insights into passenger trends, which can assist in planning and decision-making for airport management. Although the model produces reasonable results, the accuracy is limited due to the use of a simple algorithm and a restricted set of features. Overall, the project highlights the significance of big data analytics and scalable tools like PySpark in solving real-world problems in the aviation sector.

Future scope

The project can be further enhanced by incorporating advanced machine learning algorithms such as Random Forest, Gradient Boosting, or deep learning models to improve prediction accuracy. Additional features like geographic region, terminal details, flight type, and seasonal variations can be included to make the model more robust. The system can also be extended to support real-time data processing using streaming technologies for live passenger traffic prediction. Furthermore, developing an interactive dashboard using tools like Power BI or Tableau can improve visualization and user experience. Integration with airport management systems and deployment through web or mobile applications can make the solution more practical and scalable for real-world implementation.

Github link

https://github.com/sabeer708/airport-traffic-prediction